RETRIEVE '26 Workshop · MICRO-59

FlowIVF: Restructuring ANN Search for Spatial Dataflow Accelerators

Seongkyu Lee[1], Gangmin Lee[1], Sue Moon[1], Gyuyeong Kim[2]

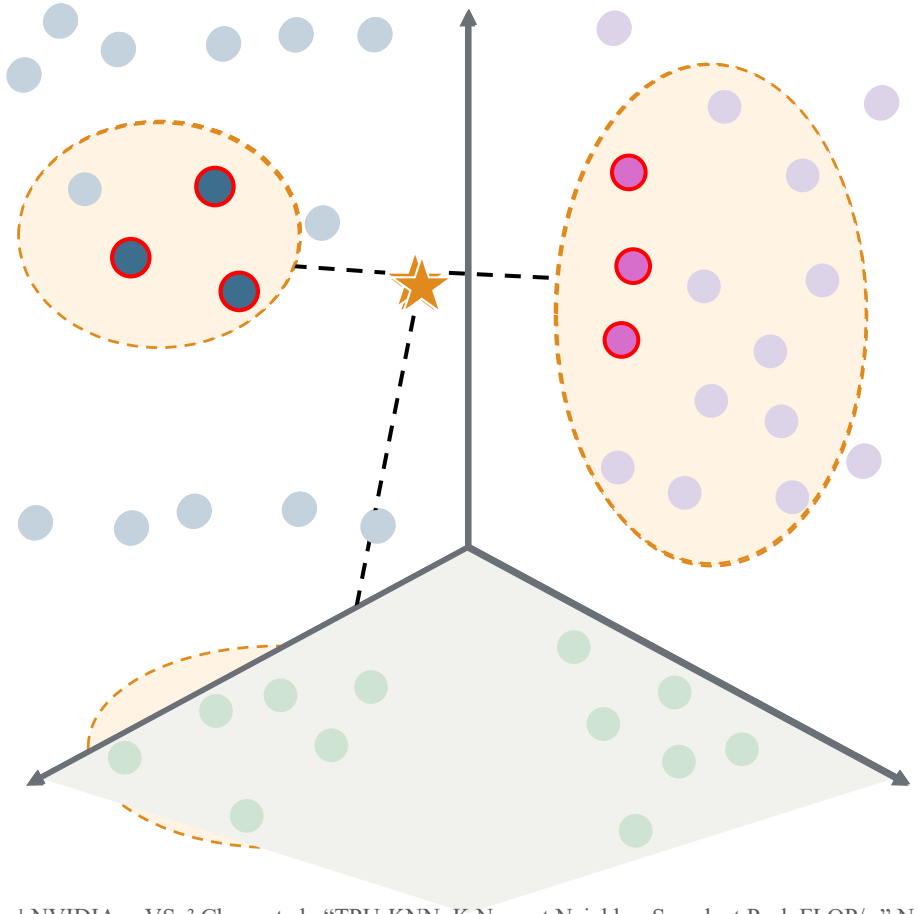
[1]KAIST School of Computing [2]Yonsei University





ANN Search: Inverted File Index (IVF)

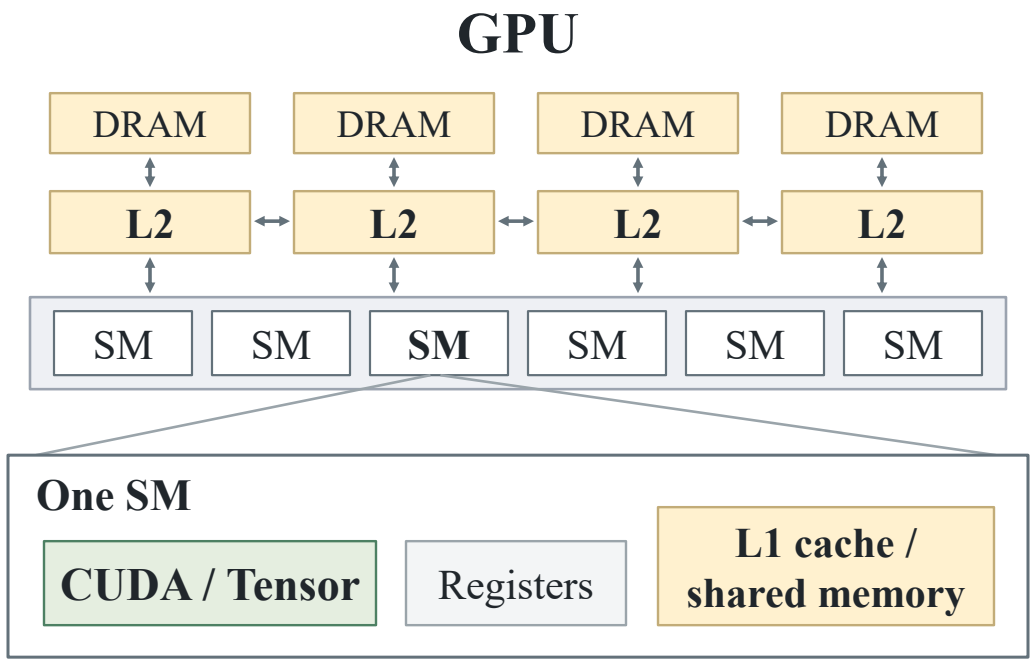
Example: nprobe = 2, K = 6



- **Approximate Nearest Neighbor (ANN) search**: core retrieval for RAG, semantic search, recommendation
- **IVF**: probe nearby lists, scan them, return top-K
  - Widely used: simple to build, easy to update, small index
- **Parallel- and batch-friendly**
  - Distance scoring accelerated on GPU[1], TPU[2], and Ascend NPU[3]

[1] NVIDIA cuVS, [2] Chern et al., "TPU-KNN: K Nearest Neighbor Search at Peak FLOP/s," NeurIPS 2022.
[3] Sun et al., "GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications," PVLDB 18(12), 2025.

GPU-based ANN Search



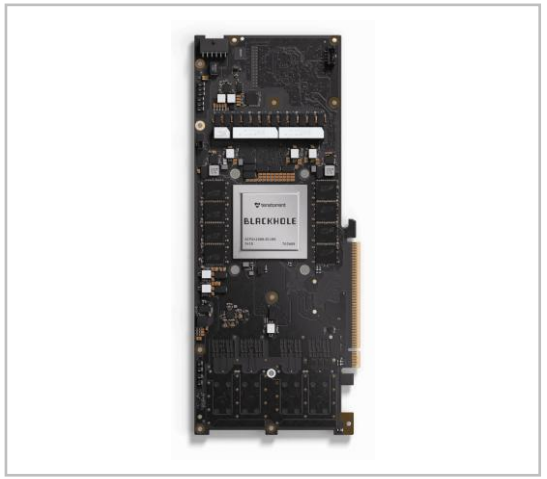
- 👍 Massive parallelism, high memory bandwidth
- 👍 Caches + HW scheduling absorb irregularity
- 👍 Mature ANNS stack (cuVS, Faiss, …)
- 👎 Memory bandwidth limits throughput
- 👎 Higher bandwidth comes at a cost

Emerging Accelerators for Lower TCO

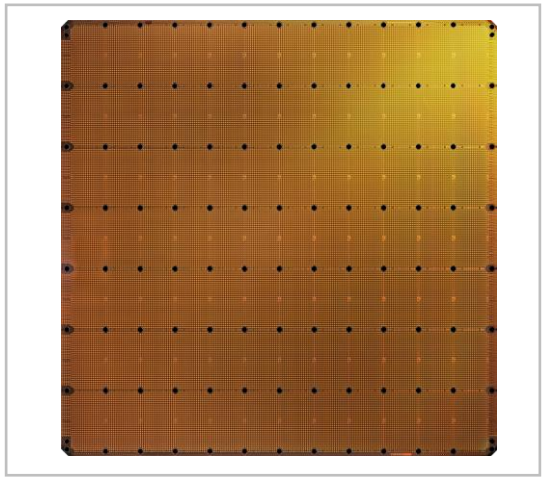
- Designed for lower TCO: higher performance per dollar and per watt
- Spatial dataflow: many cores, large distributed SRAM, Network-on-Chip (NoC)
- Already deployed for AI serving: Efficient ANN search needed (e.g., RAG)



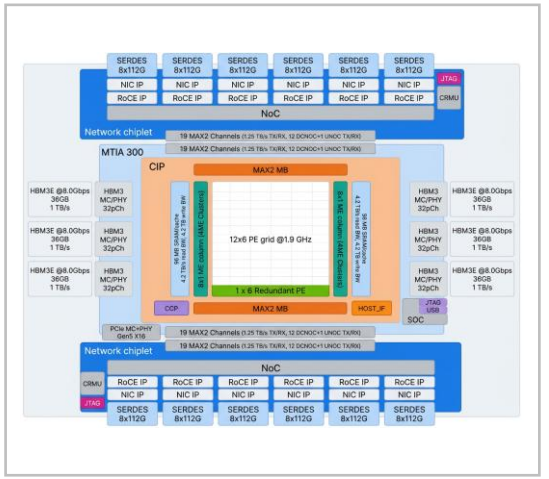
SambaNova SN40L



Tenstorrent Blackhole

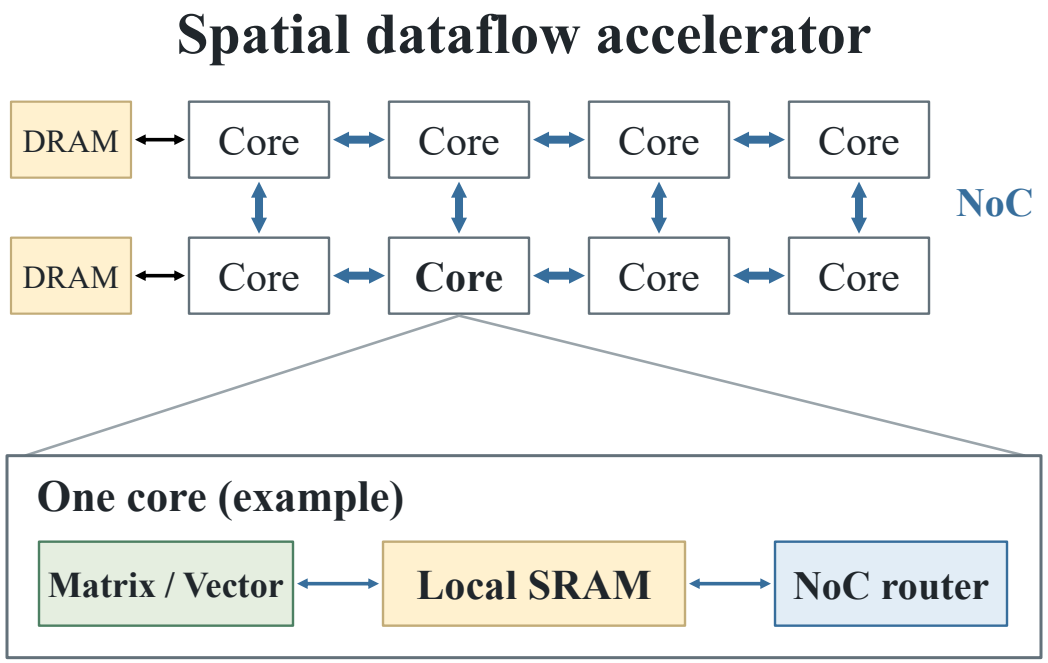


Cerebras WSE-3



Meta MTIA

NPU for ANN Search



- 👍 Lower TCO: better perf/$ and perf/W
- 👍 Large local SRAM per core
- 👍 NoC: cores exchange data without DRAM
- 👎 Explicit data movement: no shared cache
- 👎 Static mapping: work placed before execution

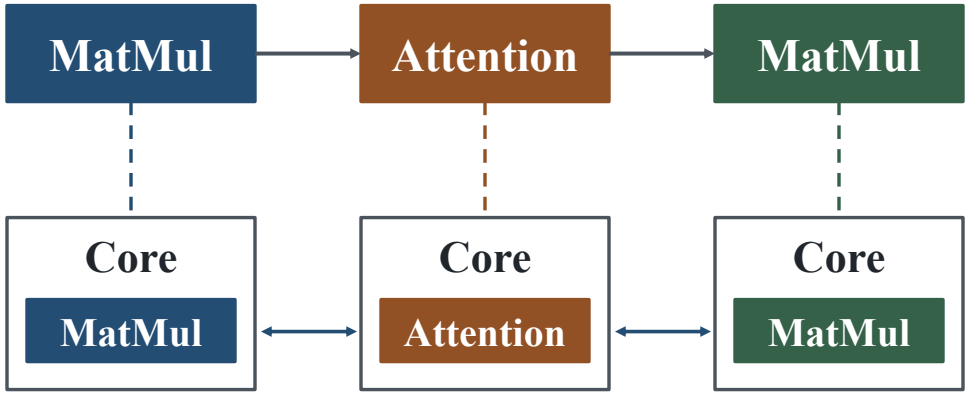
How can IVF run efficiently on spatial dataflow accelerators?

Challenge: Query-Dependent IVF Work

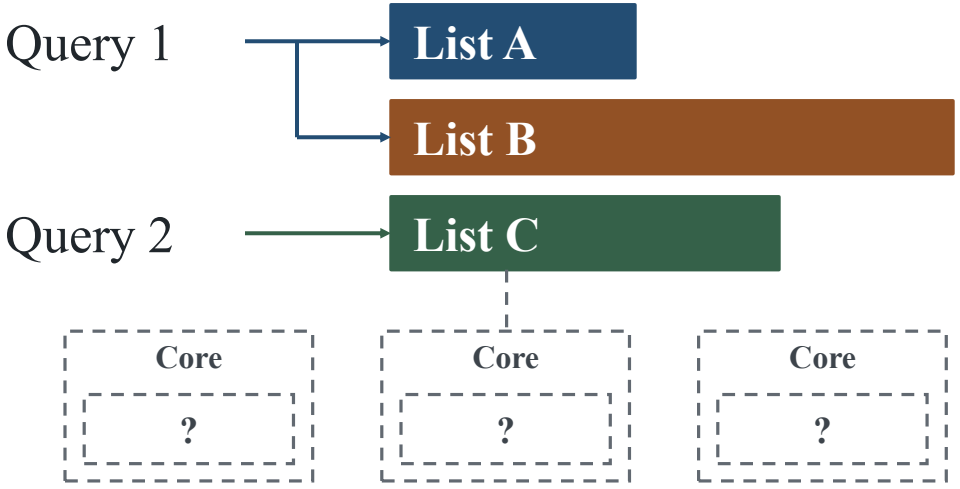
- Spatial dataflow accelerators favor a plan fixed before execution
  - LLM inference: operators and routes mapped in advance
  - IVF search: selected lists and per-core work vary by query

LLM inference

Repeated decode step (fixed-shape example)

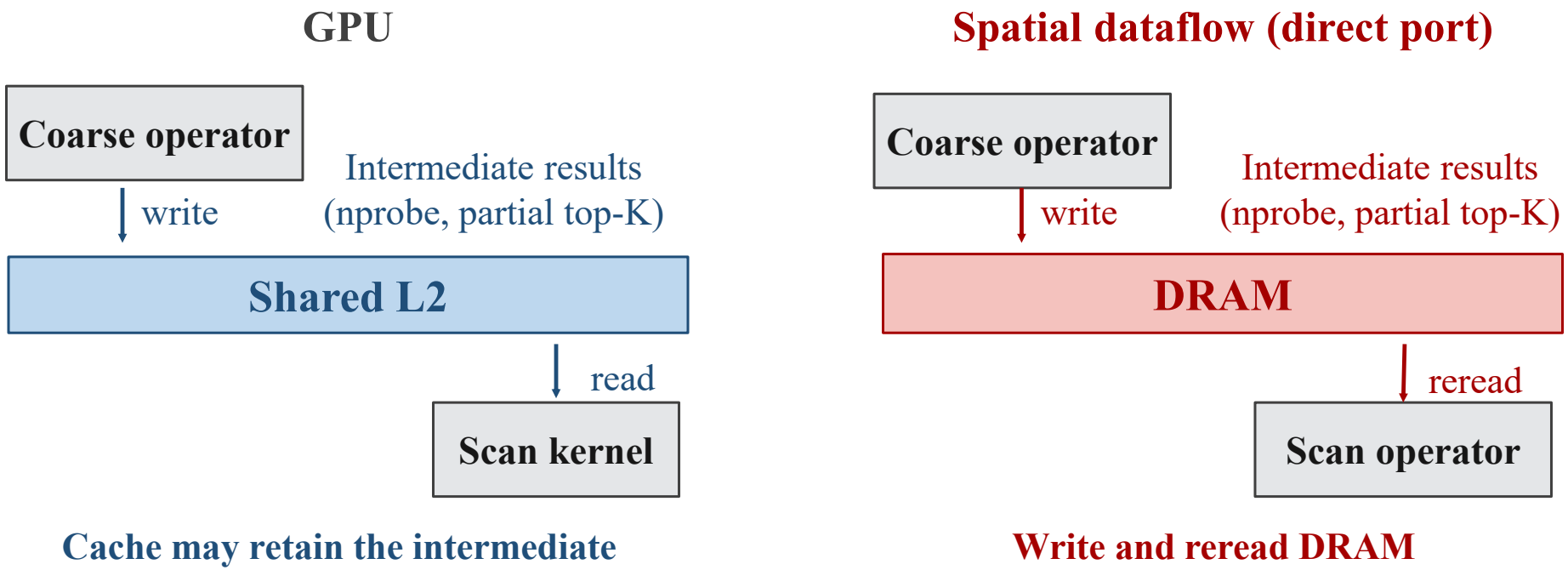


IVF execution



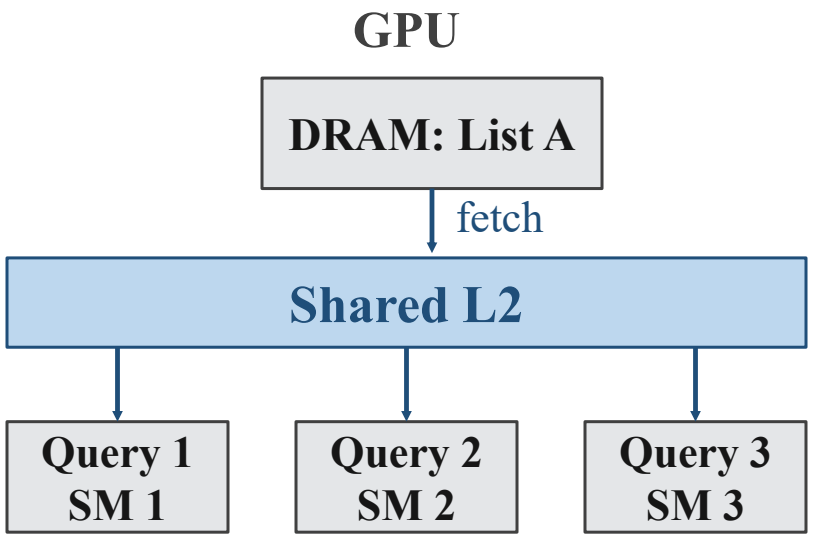
Naïve GPU-Style Porting

- GPUs provide hardware scheduling and shared caches
- Spatial dataflow requires explicit buffering and communication
  - Across stages: pass intermediate results explicitly

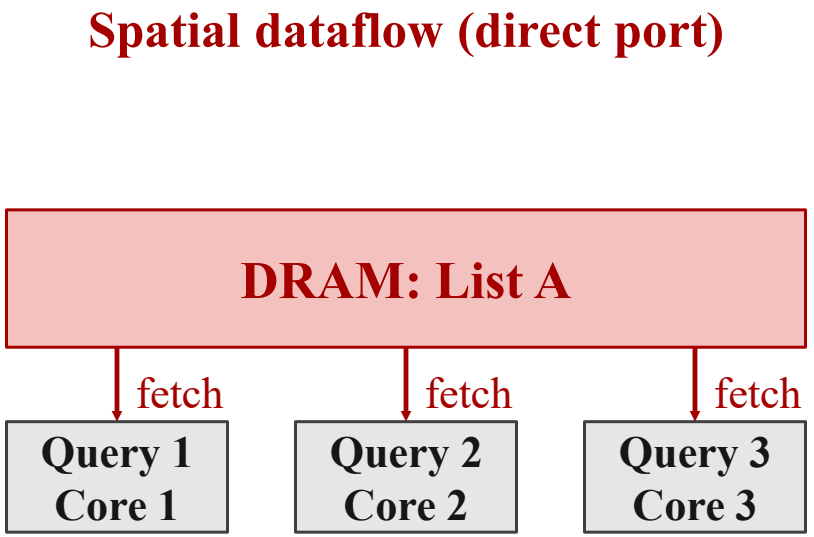


Naïve GPU-Style Porting

- GPUs provide hardware scheduling and shared caches
- Spatial dataflow requires explicit buffering and communication
  - Across stages: pass intermediate results explicitly
  - Across queries: organize page reuse explicitly



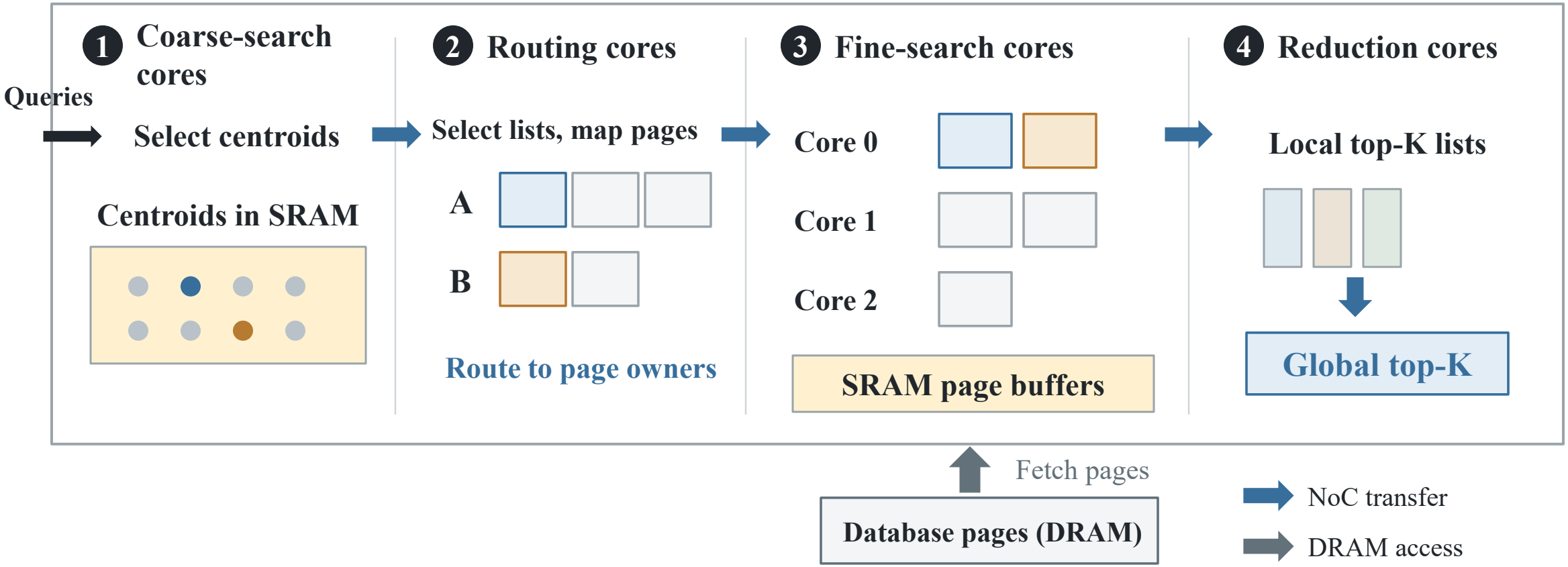
Cache hits can serve several queries



Repeated reads without explicit reuse

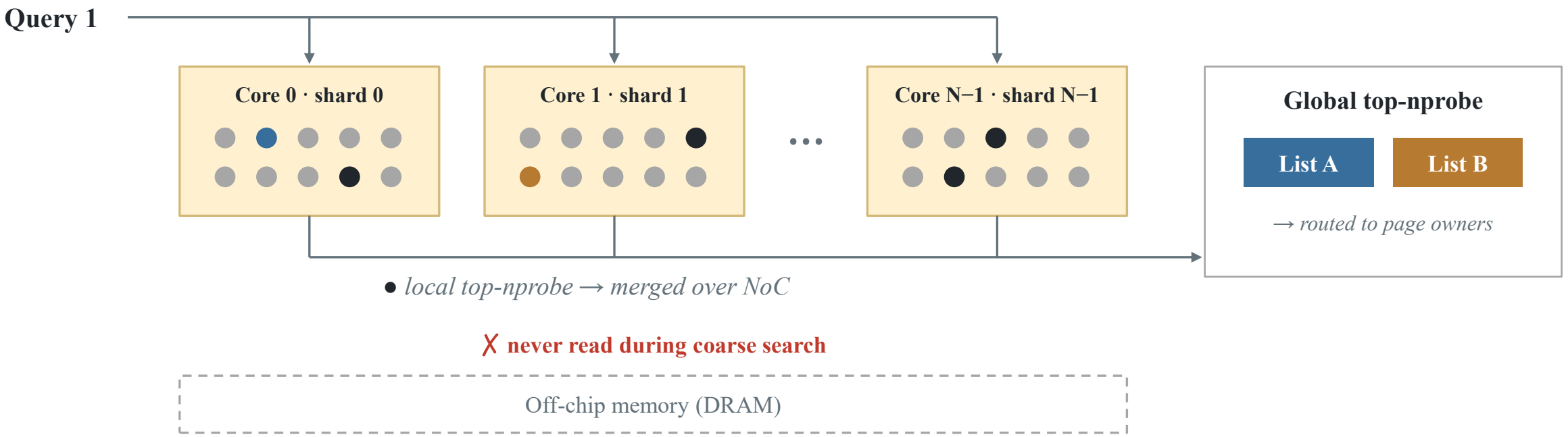
FlowIVF Design

- Intermediate results over the **NoC**, not DRAM
  - Only database pages from DRAM



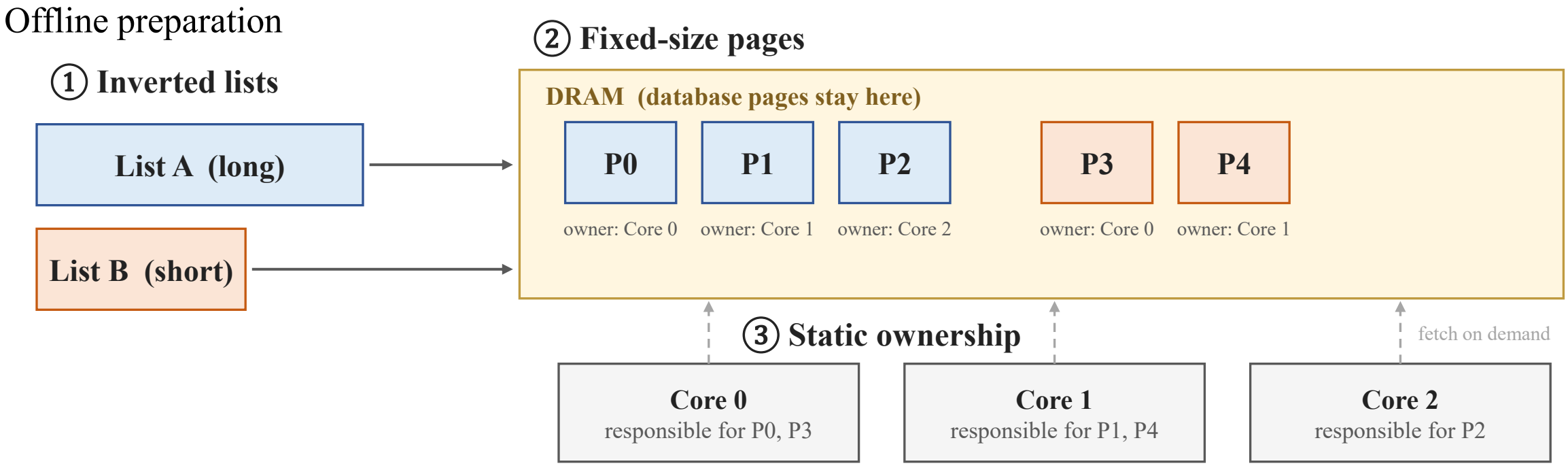
SRAM-Resident Centroids

- Centroids: small and read by every query → kept in SRAM
  - One each core compute its local top-nprobe



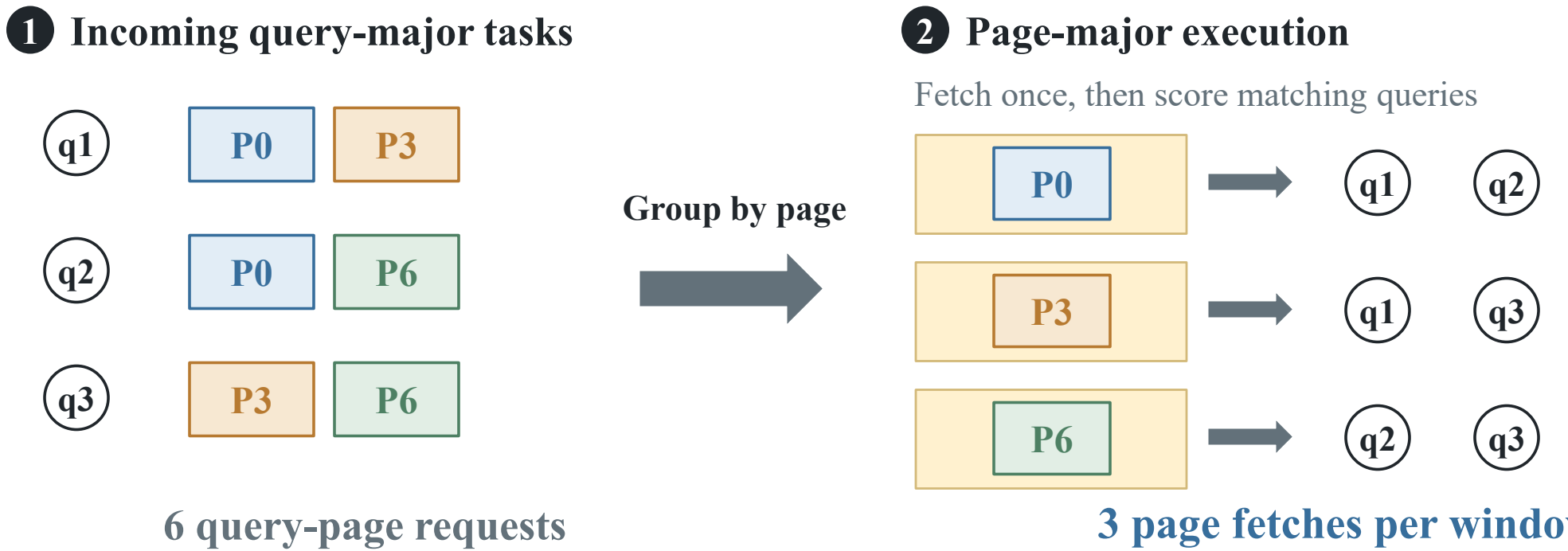
Page-Centric Reuse

- Reduce DRAM traffic through page reuse
  - Group queries within a window by page
  - Fetch each page once into local SRAM and reuse it across matching queries



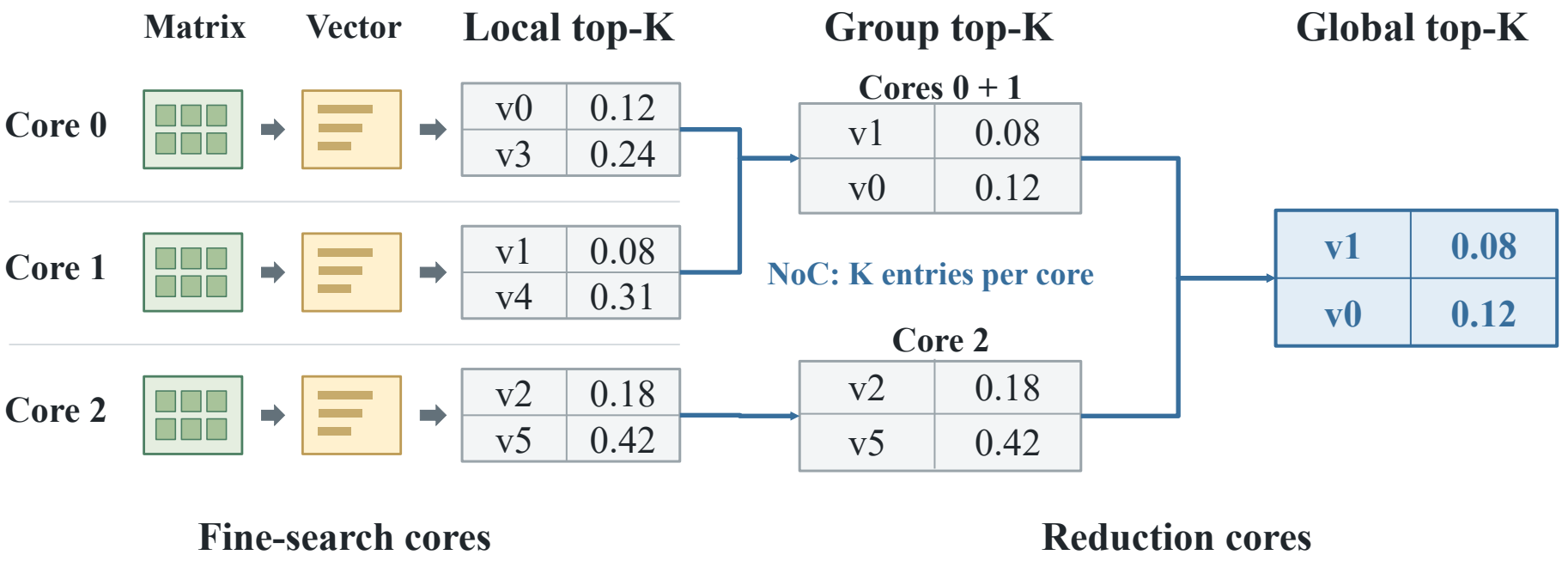
Page-Centric Reuse

- Reduce DRAM traffic through page reuse
  - Group queries within a window by page
  - Fetch each page once into local SRAM and reuse it across matching queries



Hierarchical Top-K Reduction

- Reduce communication and final-merge workload
  - Vector engines retain local top-K in SRAM
  - Reduction cores prune candidates before the final merge



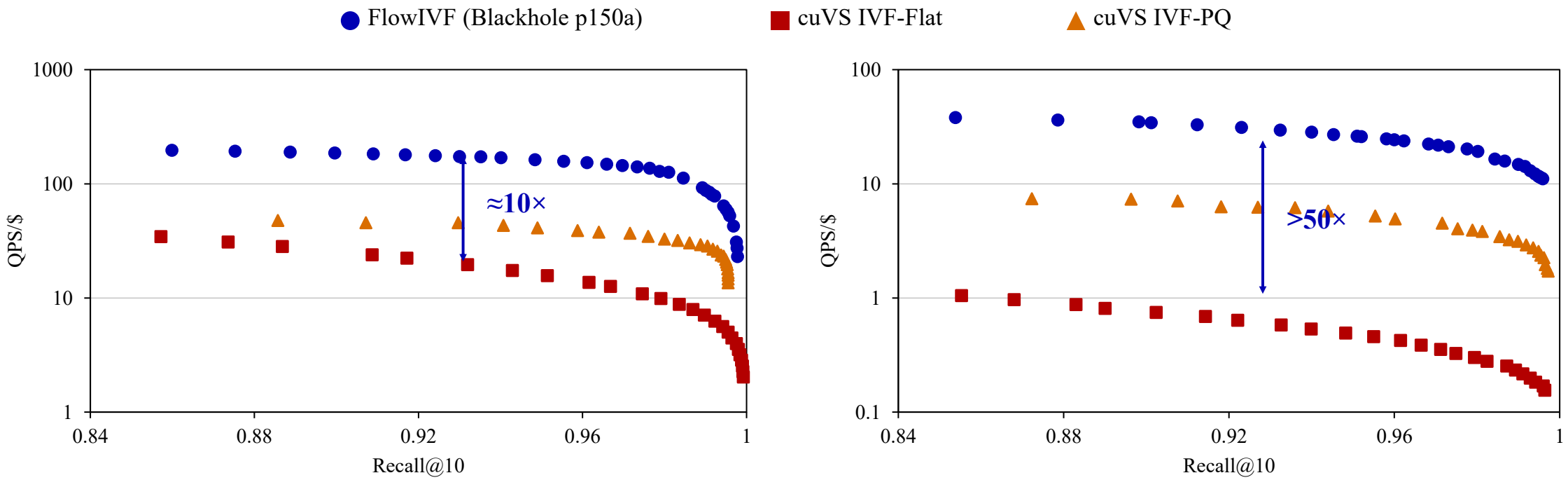
Evaluation Setup

Hardware Platforms

Spec	**Blackhole p150a**	**RTX PRO 6000 Blackwell Max-Q**
Compute units	120 Tensix cores (12 × 10)	188 SMs
SRAM	1.5 MB/core	128 KB/SM
DRAM	32 GB GDDR6	96 GB GDDR7
Off-chip BW	512 GB/s	1,792 GB/s
Purchase price	$1,399	$16,000

- Baseline: cuVS 26.02 on the GPU
  - IVF-Flat and IVF-PQ
- Vector datasets
  - SIFT-1M: 128 dimensions, 10,000 queries
  - GIST-1M: 960 dimensions, 1,000 queries

Cost Efficiency (QPS/$)

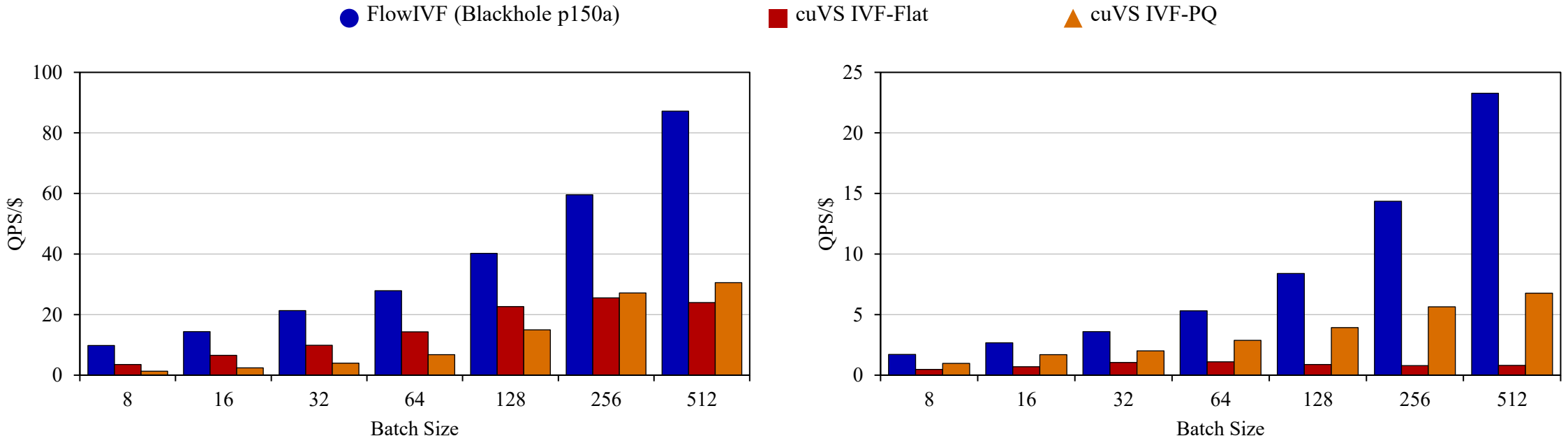


(a) SIFT-1M **(b) GIST-1M**

10× / 50× QPS/$ over IVF-Flat, 4× / 5× over IVF-PQ at Recall@10 = 0.95
Raw QPS: 1× / 5.8× IVF-Flat with 1/3 the bandwidth (SIFT / GIST)

Batch Scaling

QPS/$ vs. batch size at Recall@10 ≈ 0.90



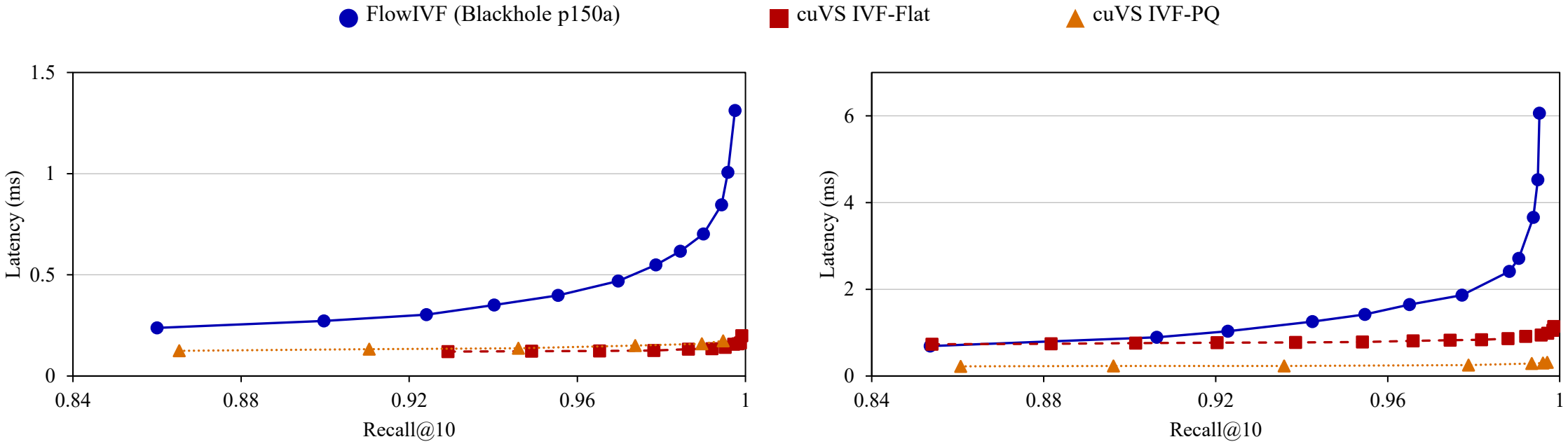
(a) SIFT-1M

(b) GIST-1M

Larger batch → more page reuse → 9× / 14× QPS/$ (batch 8 → 512)
2.9× / 3.4× the best GPU baseline at batch 512 (SIFT / GIST)

Single-Query Latency

Per-query latency at batch size 1, no cross-query reuse



(a) SIFT-1M

(b) GIST-1M

Batch 1: 0.38 ms (SIFT) / 1.4 ms (GIST) at Recall@10 = 0.95

No cross-query reuse → GPU baselines remain lower

Thank you! Questions are welcome.

FlowIVF: **cost-efficient** IVF search on **spatial dataflow accelerators**

- Restructure the **pipeline**, not the algorithm
- Three techniques for large SRAM + NoC:
  - **SRAM-resident centroids**: coarse search without DRAM
  - **Page-centric reuse**: one DRAM fetch per page per window
  - **Hierarchical top-K**: exact merge over the NoC
- **10–50× QPS/$** over GPU IVF-Flat, ms-class latency at batch 1
- **Future work:** quantization (PQ, RaBitQ), multi-chip scaling, graph-based ANN